

JNX Learning Platform - Developer Quickstart

Quick 10-Minute Guide to Adding Your Product

What is This?

The JNX Learning Platform collects data from all JNX products (Qryx, Trading Bot, etc.) to enable:

- AI-powered optimization
- Pattern recognition
- Performance monitoring
- Continuous improvement

Think of it as: A central nervous system for all JNX products.

Prerequisites

- ✓ Phase 1 is complete (SDK Foundation)
 - ✓ Database migration executed (`MIGRATION_JNX_LEARNING_PLATFORM.sql`)
 - ✓ You have a product to integrate
-

Quick Integration (3 Steps)

Step 1: Create Product Config

Create file: `lib/jnx-products/[your-product]/config.ts`

```
import { z } from 'zod'
import { defineProduct } from '@lib/jnx-core/registry'

export default defineProduct({
  id: 'your_product',           // Unique ID
  name: 'Your Product Name',    // Display name
  version: '1.0.0',

  events: {
    'event_name': {
      schema: z.object({
        // Define your event structure
        field1: z.string(),
        field2: z.number(),
      }),
      description: 'What this event tracks'
    }
  },

  protected: ['core/*', 'api/payment/*'], // AI can't touch
  optimizable: ['prompts/*', 'ui/formatting'], // AI can optimize

  goals: {
    responseTime: { target: 2000, unit: 'ms' }
  }
})
```

Step 2: Register Product

Edit: lib/jnx-products/index.ts

```
import './qryx/config'
import './your-product/config' // ← Add this line
```

Step 3: Use in Your Code

```
import { useProductLogger } from '@lib/jnx-products'

function YourComponent() {
  const logger = useProductLogger('your_product')

  const handleAction = async () => {
    await logger.logEvent('event_name', {
      field1: 'value',
      field2: 123
    })
  }

  return <button onClick={handleAction}>Do Something</button>
}
```

Done! 🎉 Your product is now integrated.

Verify It Works

1. TypeScript Check

```
cd nextjs_space  
yarn tsc --noEmit
```

Expected: No errors

2. Build Check

```
yarn build
```

Expected: Successful build

3. Database Check

```
-- In Supabase SQL Editor  
SELECT product_type, event_type, COUNT(*)  
FROM product_events  
GROUP BY product_type, event_type;
```

Expected: Your product appears with event counts

Real Example: Trading Bot

config.ts

```
import { z } from 'zod'
import { defineProduct } from '@lib/jnx-core/registry'

export default defineProduct({
  id: 'trading_bot',
  name: 'JNX Trading Bot',
  version: '1.0.0',

  events: {
    'trade_executed': {
      schema: z.object({
        symbol: z.string(),
        action: z.enum(['buy', 'sell']),
        amount: z.number().positive(),
        price: z.number().positive(),
        profit_loss: z.number(),
        strategy: z.string()
      }),
      description: 'Trade execution event'
    }
  },

  protected: [
    'core/order-execution',
    'core/risk-management'
  ],

  optimizable: [
    'strategies/entry-timing',
    'strategies/exit-timing'
  ],

  goals: {
    profitability: { target: 0.15, unit: 'percentage' },
    winRate: { target: 0.6, unit: 'percentage' }
  }
})
```

Usage

```
import { useProductLogger } from '@lib/jnx-products'

function TradingPanel() {
  const logger = useProductLogger('trading_bot')

  const executeTrade = async (trade: Trade) => {
    const result = await exchange.placeOrder(trade)

    await logger.logEvent('trade_executed', {
      symbol: trade.symbol,
      action: trade.action,
      amount: trade.amount,
      price: result.price,
      profit_loss: calculatePL(result),
      strategy: 'momentum'
    })
  }
}
```

Key Concepts

Protected Paths

AI will NEVER modify these:

- Core business logic
- Payment processing
- Authentication
- Security features
- Database operations

Optimizable Paths

AI CAN suggest improvements (with approval):

- AI prompts
- UI formatting
- Performance tuning
- Caching strategies

Goals

What AI should optimize towards:

```
goals: {
  responseTime: { target: 2000, unit: 'ms' },
  accuracy: { target: 0.95, unit: 'percentage' },
  satisfaction: { target: 4.5, unit: 'rating' }
}
```

Troubleshooting

“Product not found”

→ Did you add import in `lib/jnx-products/index.ts` ?

“Event type not defined”

→ Check your config’s `events` object

“Validation failed”

→ Event data doesn’t match Zod schema

Events not in database

→ Did you run `MIGRATION_JNX_LEARNING_PLATFORM.sql` ?

Best Practices

✓ DO:

- Use descriptive event names (`trade_executed` , not `event1`)
- Validate all fields with Zod
- Set measurable goals
- Keep schemas simple
- Test before committing

✗ DON’T:

- Modify core SDK files
 - Use `any` types
 - Log PII without redaction (automatic)
 - Skip validation
 - Set unrealistic goals
-

Advanced Features

Debug Mode

```
const logger = useProductLogger('your_product', {
  debugMode: true, // Logs to console
  enablePIIRedaction: false // Only for local testing!
})
```

Batch Logging

```
const logger = useProductLogger('your_product', {
  batchSize: 10, // Send 10 events at once
  retryAttempts: 5 // Retry 5 times on failure
})
```

Session Tracking

```
const logger = useProductLogger('your_product')
const sessionId = logger.getSessionId()

// All events share this session ID
await logger.logEvent('event1', { ... })
await logger.logEvent('event2', { ... })

// Reset session (e.g., on logout)
logger.resetSession()
```

What's Next?

Phase 2: AI Analysis Engine

- Automatic pattern detection
- AI-generated insights
- Optimization proposals

Phase 3: Admin Dashboard

- Real-time analytics
- Goal tracking
- Event visualization

Phase 4: Human-in-the-Loop

- Approval workflow for AI changes
- A/B testing
- Deployment tracking

Resources

Documentation

- `JNX_LEARNING_PLATFORM_PHASE1.md` - Complete Phase 1 guide
- `lib/jnx-products/README.md` - Detailed integration guide
- `ABACUS_AGENT_ONBOARDING.md` - For Abacus AI agents

Code Examples

- `lib/jnx-products/qryx/config.ts` - Reference implementation
- `lib/jnx-core/types.ts` - All TypeScript types

Database

- `MIGRATION_JNX_LEARNING_PLATFORM.sql` - Schema definition
-

Questions?

Check:

1. This quickstart
2. `lib/jnx-products/README.md`
3. Example config (`lib/jnx-products/qryx/config.ts`)
4. Full guide (`JNX_LEARNING_PLATFORM_PHASE1.md`)

Still stuck?

- Check TypeScript errors
 - Verify database migration
 - Review Qryx example
 - Test with debug mode
-

Version: 1.0.0

Last Updated: 2024-12-28

Status: Production-Ready 